

# 1. Beadandó feladat dokumentáció

## Készítette:

Fábián László

ZJYRR2

Email: [fabianlaszlo25@gmail.com](mailto:fabianlaszlo25@gmail.com)

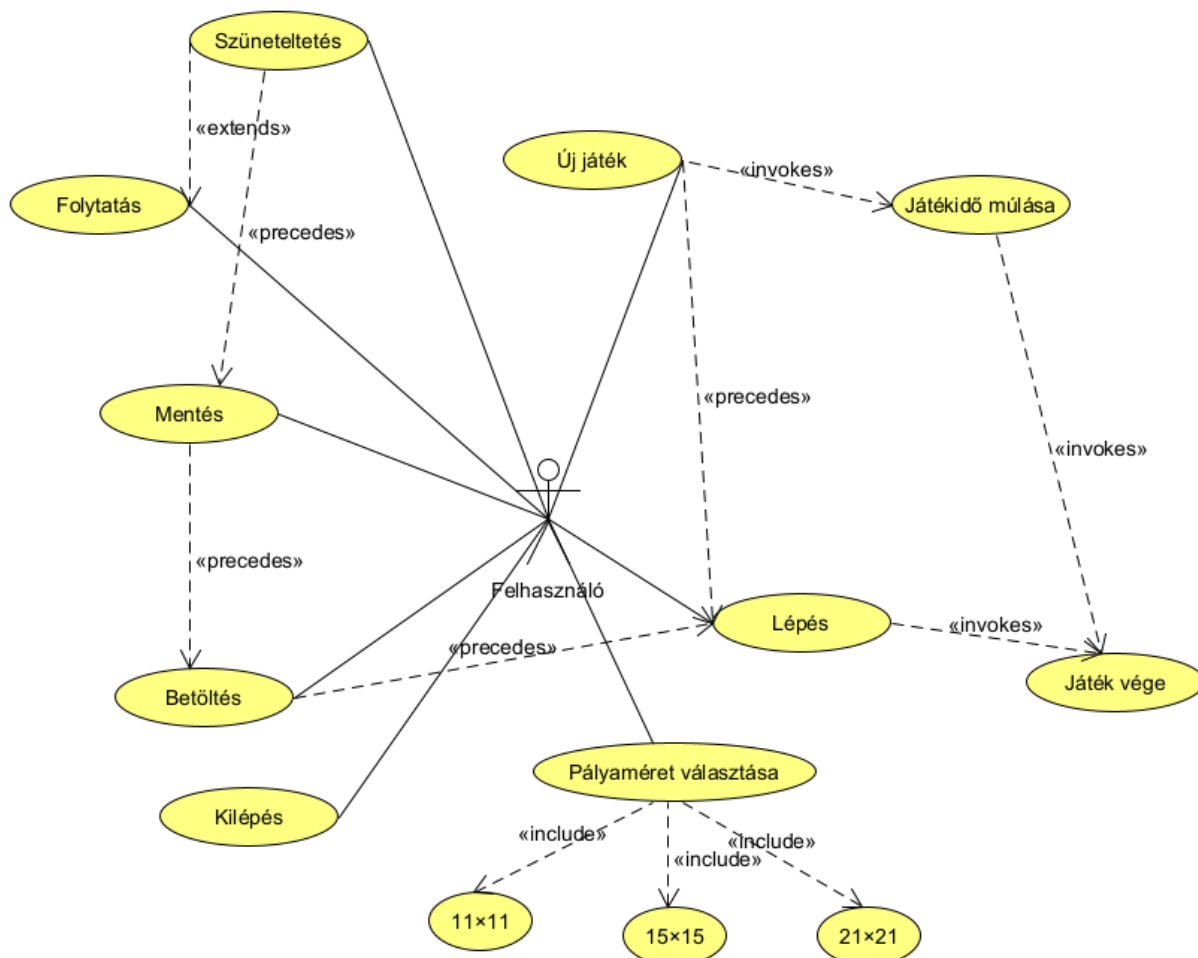
## Feladat: Menekülj

Készítsünk programot, amellyel a következő játékot játszhatjuk. Adott egy  $n \times n$  elemből álló játékpálya, ahol a játékos két üldöző elől próbál menekülni, illetve próbálja őket aknára csalni. Kezdetben a játékos játékpálya felső sorának közepén helyezkedik el, a két üldöző pedig az alsó két sarokban. Az ellenfelek adott időközönként lépnek egy mezőt a játékos felé haladva úgy, hogy ha a függőleges távolság a nagyobb, akkor függőlegesen, ellenkező esetben vízszintesen mozognak a játékos felé. A pályán véletlenszerű pozíciókban aknák is elhelyezkednek, amelyekbe az ellenfelek könnyen beleléphetnek, ekkor eltűnnek (az akna megmarad). A játékos vízszintesen, illetve függőlegesen mozoghat (egyesével) a pályán, és célja, hogy az ellenfeleket aknára csalja, miközben ő nem lép aknára. Ha sikerül minden üldözőt aknára csalnia, akkor győzött, ha valamely ellenfél elkapja (egy pozíciót foglal el vele), vagy aknára lép, akkor veszített. A program biztosítson lehetőséget új játék kezdésére a pályaméret megadásával ( $11 \times 11$ ,  $15 \times 15$ ,  $21 \times 21$ ), valamint játék szüneteltetésére (ekkor nem telik az idő, és nem léphet senki). Ismerje fel, ha vége a játéknak, és jelenítse meg, hogy győzött, vagy veszített-e a játékos. Ezen felül szüneteltetés alatt legyen lehetőség a játék elmentésére, valamint betöltésére. A program játék közben folyamatosan jelezze ki a játékidőt.

## Elemzés

- A játék célja, hogy a játékos egy  $n \times n$  méretű négyzetrácson két üldöző elől meneküljön, és megpróbálja őket aknára csalni, mielőtt azok elkapnák. A játékos akkor veszít, ha aknára lép, vagy ha valamelyik üldöző eléri őt. Ha mindkét üldöző aknára lép, a játékos győz.
- A feladatot egyablakos asztali alkalmazásként Windows Forms grafikus felülettel valósítjuk meg.
- A játékot három különböző pályamérettel lehet játszani:  **$11 \times 11$** ,  **$15 \times 15$**  és  **$21 \times 21$** . A program indításkor az alapértelmezett pályaméret  **$11 \times 11$** , de a felhasználó új játék indítása előtt szabadon módosíthatja ezt.
- Az ablak tartalmazza a játéktér kirajzolását, a vezérlőgombokat (*Új játék*, *Szünet/Folytatás*, *Mentés*, *Betöltés*), a pályaméret-választót, valamint az eltelt idő kijelzését.

- A program esemény vezérelt módon kezeli a játékos irányítását: a játékos a **A, W, S, D** segítségével léptethető, minden gombnyomás egyetlen mezőnyi mozgást eredményez. Hosszan tarva gyorsabban is lehet mozogni.
- A pálya elemei vizuálisan megkülönböztetettek:
  - **zöld**: játékos,
  - **piros**: üldöző,
  - **fekete**: akna,
  - **fehér**: üres mező.
- A játék **győzelmi és veszteségi feltételeit** a logikai modell folyamatosan figyeli. Ha az összes üldöző aknára lépett, győzelmi üzenet jelenik meg; ha a játékos aknára lép vagy ellenséghez ütközik, a program vereséget jelez.
- A program **támogatja a játék szüneteltetését**. Szünet esetén a játékmenet leáll, az időzítők inaktívak lesznek, és a képernyő tetején megjelenik a „SZÜNET” felirat. A szüneteltetés ideje alatt elérhető a **mentés és betöltés** funkció.
- A játék végén a program **felugró üzenetablakkal** tájékoztatja a játékost az eredményről („Győzelem” vagy „Vereség”), majd visszatér szüneteltetett állapotba, lehetőséget adva új játék indítására.



**Felhasználói esetek Diagramja:**

## Tervezés

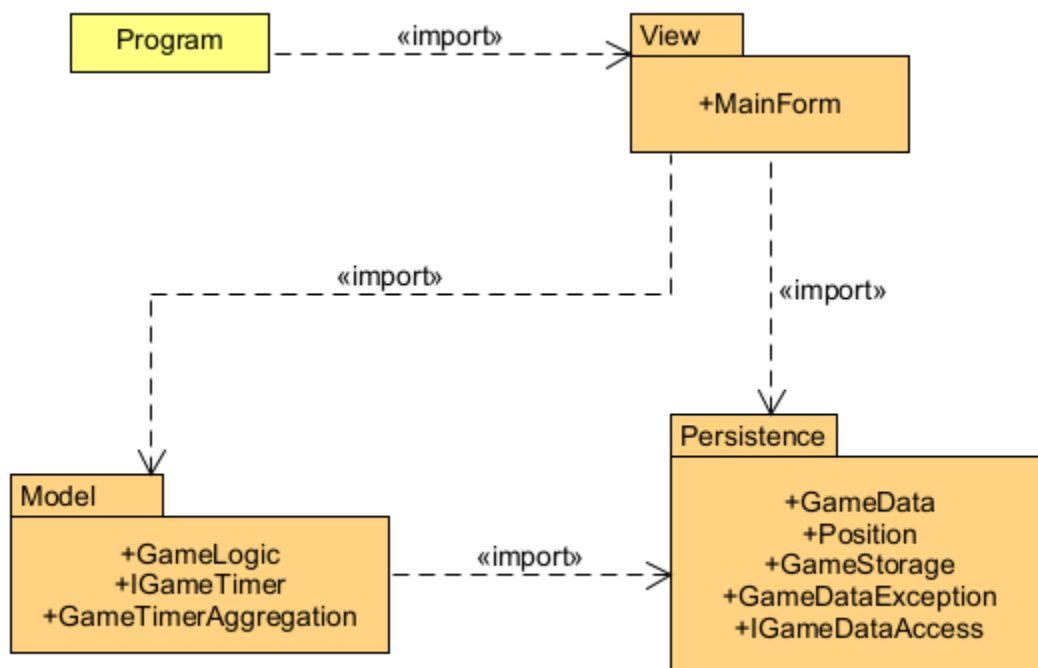
### • Programszerkezet:

- Programszerkezet:
- A programot háromrétegű architektúrában valósítjuk meg. A megjelenítés a **View**, a játéklógika a **Model**, míg az adattárolás a **Persistence** névtérben helyezkedik el.
- A program szerkezetét három fő névtérre osztjuk:
- **Menekulj.Model** - tartalmazza a játék üzleti logikáját, játékállapot-kezelést és időzítő funkciókat
- **Menekulj.Persistence** - felelős a játékállapot tartós tárolásáért és visszatöltéséért
- **Menekulj.WinForms** - implementálja a Windows Forms alapú felhasználói felületet

---

### • Perzisztencia:

- Az adatkezelés feladata a "Menekülj" játék állapotinformációinak tárolása, valamint a betöltés/mentés biztosítása.
- A **GameData** osztály reprezentálja a játék menthető állapotát, amely tartalmazza:
- A pálya méretét (N - int)
- A játékos aktuális pozícióját (Player - Position)
- Az üldözők listáját (Enemies - List<Position>)
- Az aknák listáját (Mines - List<Position>)
- Az eltelt játékidőt (ElapsedSeconds - int)
- A hosszú távú adattárolás lehetőségeit az **IGameDataAccess** interfész adja meg, amely lehetőséget ad a játékállapot mentésére (Save) és betöltésére (Load).
- Az interfészt **JSON** fájl alapú adatkezelésre a **GameStorage** osztály valósítja meg. A fájlkezelés során fellépő hibákat a **GameDataException** kivétel jelzi.
- A program az adatokat JSON formátumban tárolja el, melyek a **savegame.json** fájlba kerülnek. A fájl szerkezete ember által olvasható, és tartalmazza a játék összes releváns állapotinformációját.



**Az Alkalmazás csomagdiagramja:**

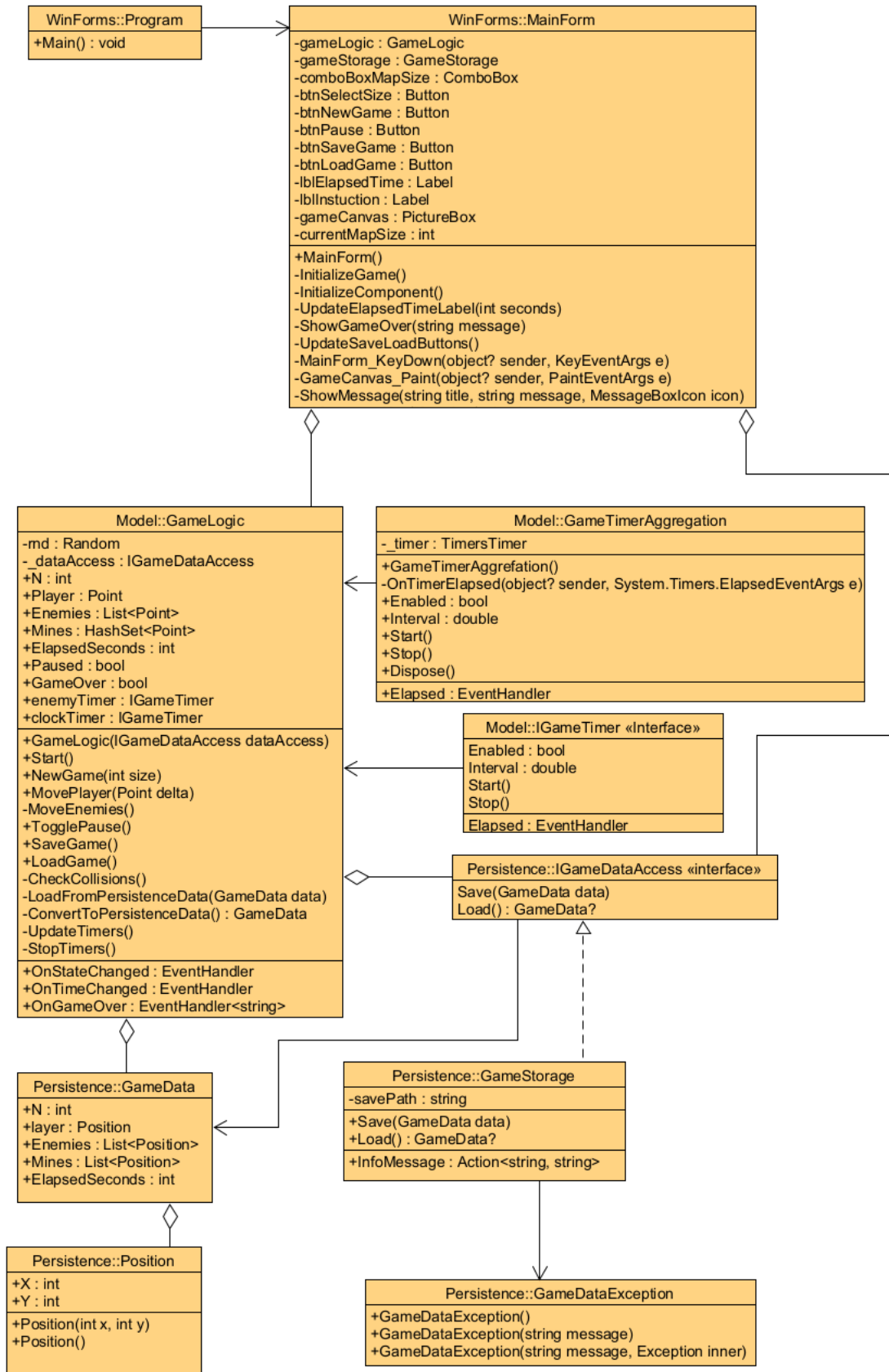
#### • Modell:

- A játék logikáját a **GameLogic** osztály valósítja meg, amely kezeli a játéktér állapotát, a mozgásokat és az eseményeket. A modell tárolja a játék aktuális adatait:
- a pályaméretet (N),
- a játékos pozícióját (Player),
- az üldözők listáját (Enemies),
- az aknák halmazát (Mines),
- az eltelt időt és a játékmenet állapotát (Paused, GameOver).
- A játék indítását a **NewGame()** metódus végzi, amely inicializálja az összes entitást: a játékos a pálya tetején középen helyezkedik el, az üldözők az alsó két sarokban indulnak, az aknák pedig véletlenszerű helyre kerülnek, ahol nincs más objektum.
- A **mozgások kezelése** a **MovePlayer()** és **MoveEnemies()** metódusokon keresztül történik. A játékos mozgását a felhasználó billentyűleütései váltják ki, míg az üldözők mozgása automatikusan, az időzítő eseményei alapján történik. Az üldözők minden lépésben a játékos felé közelítenek a legrövidebb úton, felváltva vízszintesen és függőlegesen haladva.
- A **győzelem és vereség feltételeit** a modell ellenőrzi minden lépés után a **CheckCollisions()** metódusban.
- Az **eseménykezelés** a modellben eseménydelegáltakkal valósul meg:
- **OnStateChanged** -- a pálya állapotának frissítése,
- **OnTimeChanged** -- az időzítés kezelése,
- **OnGameOver** -- a játék befejeződése.

- Ezek az események lehetővé teszik, hogy a View automatikusan reagáljon a modell változásaira.
- Az időzített eseményeket két Timer objektum biztosítja: az egyik az **idő számlálásáért**, a másik az **ellenfelek mozgásáért** felel. Szüneteltetéskor mindkettő megáll, folytatáskor újraindul.

#### Nézet:

- A nézetet a **MainForm** osztály biztosítja, amely tárolja a modell egy példányát (**gameLogic**), valamint az adatelérés konkrét példányát (**gameStorage**).
- A játéktér megjelenítését egy **PictureBox (gameCanvas)** végzi, amely a **GameCanvas\_Paint** eseménykezelőben rajzolja ki a pályát, a játékost, az üldözőket és az aknákat.
- A felületen elhelyezett vezérlőelemek:
- **Pályaméret választó** (comboBoxMapSize) - 11×11, 15×15, 21×21 opciók
- **Műveletgombok** - Új játék, Szünet/Folytatás, Mentés, Betöltés
- **Időmegjelenítő** (lblElapsedTime) - eltelt játékidő másodpercben
- A felhasználói interakciók kezelése:
- **Billentyűzet események** - játékos mozgatása (WASD/nyílbillentyűk)
- **Gomb események** - játékvezérlés és fájlműveletek
- **Eseményfeliratkozások** - modell állapotváltozásainak kezelése



## Az Alkalmazás Osztály-diagramja:

### Tesztelés

- A modell funkcionalitása egységtesztek segítségével lett ellenőrizve a **GameLogicTests** osztályban.
- A tesztelés célja a játékmenet fő logikai elemeinek helyes működésének vizsgálata, különös tekintettel a mozgás, ütközéskezelés, időkezelés és állapot-változás folyamatára.
- Az időzítő funkcionalitás teszteléséhez egy **MockTimer** osztályt implementáltunk, amely lehetővé teszi az időzítő eseményeinek manuális kiváltását tesztkörnyezetben.
- Az alábbi tesztesetek kerültek megvalósításra:
- **GameLogicNewGameTest:** Új játék indítása, a játékos, üldözők és aknák helyes inicializálásának ellenőrzése. Ellenőrzi, hogy a játékos a pálya közepén felül, az üldözők az alsó sarkokban helyezkednek-e el, és legalább egy akna legyen jelen.
- **GameLogicMovePlayerTest:** A játékos mozgásának tesztelése. Ellenőrzi, hogy a játékos helyesen mozog-e a megadott irányba, és a pozíció frissül-e.
- **GameLogicShouldNotMoveOutsideBoard:** A pályahatárok kezelésének vizsgálata. Bizonyítja, hogy a játékos nem hagyhatja el a pálya határait.
- **GameLogicEnemyMoveTest:** Az üldözők mozgásának tesztelése. Ellenőrzi, hogy az üldözők tényleg mozognak-e az időzítő esemény hatására.
- **GameLogicPauseTest:** A szüneteltetés és folytatás funkciók helyes működésének ellenőrzése. Bizonyítja, hogy a TogglePause() metódus helyesen vált a szünet állapot között.
- **GameLogicGameOverByMineTest:** Az aknára lépés esetén történő játékbefejezés vizsgálata. Szimulálja a játékos aknára lépését és ellenőrzi, hogy a játék véget ér-e.
- **GameLogicGameOverByEnemyTest:** Az üldöző által való elkapás esetén történő játékbefejezés tesztelése. Szimulálja a játékos és üldöző ütközését.